Programación de Inteligencia Artificial

Francisco Javier Sueza Rodríguez

16 de octubre de 2026

Centro: IES Zaidin-Vergeles
Curso: Inteligencia Artificial y Big Data
Asignatura: Programación de Inteligencia Artificial

Índice

1 Introducción a la Programación de Inteligencia Artificial **4**
1.1 Introducción a la Programación 4
1.1.1 Clasificación de los Lenguajes de Programación 5
1.1.2 Ciclo de Desarrollo del Software 7
1.1.2.1 Machine Learning 9
1.2 Programación de Inteligencia Artificial 10
1.2.1 Elección del Lenguaje de Programación para IA 11
1.2.2 Lenguajes de Marcas e IA 12

Glosario **13**

Referencias **14**

Índice de figuras

1.1 Ciclo de desarrollo del software 8
1.2 Etapas en el entrenamiento de un modelo de ML 9

Índice de tablas

1. Introducción a la Programación de Inteligencia Artificial

En esta era tecnológica, la Inteligencia Artificial está en todas partes transformando radicalmente como vivimos. Desde la automatización de proceso, hasta la toma de decisiones complejas, la IA se esta convirtiendo en un pilar fundamental en diversos campos. En este primer tema, exploraremos los fundamentos de programación de la IA, desde conceptos básicos de programación hasta consideraciónes propias de la IA.

También se presentarán los distintos perfiles profesionales relacionados con el desarrollo de la IA, así como las consideraciones éticas en su desarrollo y uso.

1.1. Introducción a la Programación

La **programación** es el proceso de diseñar un algoritmo pare resolver un problema concreto y traducirlo a instrucciones que un computador pueda interpretar y ejecutar. Un **algoritmo** es una secuencia lógica de instrucción diseñada para resolver un problema concreto, de manera efectiva, eficiente y precisa.

Un **buen programador** debe tener ciertos conocimientos básicos y reunir ciertas características para desarrollar su trabajo de forma eficiente. Entre estos conocimientos y características podemos encontrar los siguientes:

- **Capacidad de Abstracción**: es una habilidad básica para todo programador y consiste en la separación de un problema de su contexto para analizarlo y dividirlo en problemas más básicos para su resolución, encontrando similitudes con otros problemas durante el proceso y reutilizando soluciones independientemente del contexto si es necesario.
- **Pensamiento Lógico**: es la capacidad de encontrar relaciones entre objetos, reales o abstractos, y llegar a conclusiones basándose en principios y reglas. De una forma práctica, es la capacidad para desarrollar algoritmos eficientes para resolver problemas concretos.
- **Creatividad y Simplicidad**: un programador debe ser capaz de encontrar soluciones innovadores y eficientes para los desafíos planteados, manteniendo la simplicidad en las soluciones implementadas, lo que facilitará la comprensión y el mantenimiento del programa.
- **Conocimientos Técnicos**: un buen programador tendrá que conocer diferentes lenguajes de programación, así como tener conocimientos sobre estructuras de datos y algoritmos, así como el dominio de entornos de desarrollo y plataformas en la nube.
- **Habilidad para Depurar Errores**: es la capacidad para asilar, identificar y corregir los errores en el código de forma sistemática y eficiente.
- **Conocimientos Matemáticos**: en del desarrollo de software de inteligencia artificial es imprescindible tener sólidos fundamentos en lógica, álgebra y estadística:
    - **Lógica Matemática**: proporciona las reglas para determinar la validez de argumentos y se aplica en programación
    - **Álgebra**: trabaja con estructuras numéricas y es esencial para realizar operaciones en técnicas de aprendizaje automático.
    - **Estadística**: se encarga de organizar y analizar datos, antes de aplicar operaciones avanzadas en aprendizaje automático.

- **Capacidad de Trabajo en Equipo**: habilidad para colaborar con otros programadores, estableciendo canales de comunicación adecuados, sabiendo defender propuestas y reconociendo el talento. Todo ello articulado gracias al uso de **software de control de versiones**.

1.1.1. Clasificación de los Lenguajes de Programación

Un **lenguaje de programación** es un conjunto de reglas y sintaxis que permite a los programadores comunicarse con un ordenador para darle instrucciones y realizar tareas específicas. Esta compuesto de un conjunto de palabras clave, estructuras y convenciones que permiten implementar programas de software de forma organizada y eficiente.

Los lenguajes de programación se utilizan para construir todo de tipo de aplicaciones y soluciones tecnológicas. Cada lenguaje de programación tiene sus características, ventajas y desventaja, pudiendo ser utilizados para resolver diferentes tipos de problemas y desarrollar diferentes tipos de aplicaciones.

Los lenguajes de programación de pueden **clasificar** atendiendo a una variedad de propiedades y características que podemos resumir en las siguientes:

1. **Nivel de Abstracción**:
   - **Lenguajes de Bajo Nivel**: son aquellos cuya sintaxis está mas cercana al lenguaje máquina y son muy dependientes del hardware donde se ejecutará el programa. Algunos ejemplos pueden ser **MIPS** y **Motorola 68000**, aunque también hay otros lenguajes más conocidos como **C** o **C++**.
   - **Lenguajes de Alto Nivel**: son lenguajes cuyo sintaxis se parece más al lenguaje natural que al lenguaje máquina, ofreciendo un nivel mayor de abstracción, lo que los hace más fáciles de entender y usar. Permiten un desarrollo más rápido a costa de menor eficiencia. Algunos ejemplos de estos lenguajes son **Python**, **Java** o **Javascript**.
2. **Modo de Ejecución**:
   - **Lenguajes Compilados**: el código fuente escrito en estos programas se traduce a lenguaje máquina antes de su ejecución. Este proceso de traducción se realiza mediante un compilador, que genera un archivo ejecutable que puede ser ejecutado directamente por la computadora. Gracias a este proceso, estos lenguajes de programación suelen tener un tiempo de ejecución más rápido y mejor rendimiento. Algunos ejemplos son **C**, **C++**, **Rust** y **Swift**.
   - **Lenguajes Interpretados**: en estos lenguajes el código fuente es interpretado y ejecutado línea por línea mediante un intérprete en tiempo de ejecución. El interprete lee y ejecuta el código sin necesidad de generar un ejecutable. Este tipo de lenguajes dependen menos del hardware donde se ejecutan, siendo más portables y flexibles. Algunos lenguajes interpretados son **Python**, **R**, **Perl** o **PHP**.
   - **Lenguajes Híbridos**: algunos lenguajes pueden reunir características de estos dos tipos. Por ejemplo, **Java** se compila a un código intermedio llamado bytecode que posteriormente se ejecuta en la máquina virtual (JVM).
3. **Paradigma de Programación**:
   - **Lenguajes de Programación Imperativa**: estos lenguajes se centran en describir como debe realizarse las operaciones paso a paso, como **C** o **Python**.

- **Lenguajes de Programación Declarativa**: estos lenguajes se centran en describir que debe hacerse, sin especificar el cómo. El mayor ejemplo de este paradigma es **Prolog**, y aunque no sean lenguajes de programación, **SQL**, **CSS** o **HTML**.
- **Lenguajes de Programación Orientada a Objetos**: estos lenguajes organizan el software en objetos, que son unidades que pueden contener los datos y métodos. Pueden usar características como la herencia o el polimorfismo. Algunos ejemplos son **Java**, **C++** o **Ruby**.
- **Lenguajes de Programación Orientada a Eventos**: estos lenguajes están basados en el manejo de eventos y en la respuesta a acciones que lleva a cabo el usuario, por ejemplo como **VistualBasic**.
- **Lenguajes de Programación Funcional**: es un caso específico de la programación declarativa basado en el uso de verdaderas funciones matemáticas y que hace use intensivo del cálculo lambda [1]. Algunos ejemplos son **Haskell**, **Lisp** o **Erlang**.
- **Lenguajes Multiparadigma**: son lenguajes que pueden admiten múltiples paradigmas de programación, dejando que sea el programador que decida cual es el enfoque más adecuado. Ejemplos son **Python**, **Javascript** o **C++**

4. **Propósito del Lenguaje**:
   - **Lenguajes de Uso Específico**: son lenguajes que están diseñados para propósitos específicos, como le procesamiento de datos (**R**) o el desarrollo web (**CSS/HTML**).
   - **Lenguajes de Uso General**: son lenguajes que se pueden emplear para desarrollar diferentes tipos de aplicaciones, como **Java**, **Python** o **C**.
5. **Sistema de Tipado**:
   - **Lenguajes de Tipado Estático**: en estos lenguajes el tipo de cada variables debe ser declarado explícitamente y verificado antes de que el programa se ejecute, verificándose dichos tipos en tiempo de compilación. Esto hace que estos lenguajes sean más robustos y seguros. Algunos ejemplos son **C** o **Java**.
   - **Lenguajes de Tipado Dinámico**: en estos lenguajes la variables pueden tomar cualquier valor tipo de dato en cualquier momento, pudiendo cambiar de tipo durante la ejecución, ya que los tipos se verifican en tiempo de ejecución. Este tipo de lenguajes ofrecen mayor simplicidad y flexibilidad en el desarrollo. Algunos ejemplos son **Python**, **Ruby** o **Javascript**.
6. **Rigidez del Tipado**:
   - **Lenguajes de Tipado Fuerte**: los lenguajes de tipado fuerte establecen fuertes restricciones sobre como se pueden hacer la conversión de tipos y se requiere una conciencia exacta de los tipos para las operaciones. Son ejemplos **Java** o **C** donde las conversiones de tipo deben declararse explícitamente.
   - **Lenguajes de Tipado Débil**: en los lenguajes de tipado débil las conversiones entre tipos son menos estrictas, permitiendo la conversión implícita entre diferentes tipos de datos y la conversión automática en determinadas circunstancias. Algunos ejemplos son **Javascript** o **PHP**.

Como vemos, hay una gran variedad de lenguajes de programación cada uno con sus características. La **elección de un lenguaje** es un aspecto crucial para que un proyecto llegue a buen termino y que dependerá, entre otros factores, de al naturaleza del proyecto, la experiencia del equipo o de la plataforma donde se implantará el software.

1.1.2. Ciclo de Desarrollo del Software

El **desarrollo de software** es el proceso de crear aplicaciones informáticas utilizando un proceso sistemático. En un campo fundamental de la informática que abarca diversas actividades orientadas a construir soluciones informáticas que satisfagan las necesidades específicas de los usuarios u organizaciones.

Este proceso, se compone de **diferentes etapas** que vamos a pasar a enumerar a continuación:

1. **Definición del Problema**: esta primera fase consiste en definir, de forma clara y precisa, el problema que se pretende resolver, definiendo el marco del problema y los objetivos a alcanzar.
2. **Análisis de Requisitos**: en esta fase de identifican y documentan los requisitos del software. Esto implica conocer las necesidades de los usuarios finales y definir las funcionalidades y características que debe tener el software. Se creará un listado exhaustivo con todos los requisitos que representa un contrato entre cliente y desarrollador, a la vez que permite al cliente que lo revise y valide, antes de pasar a otras fase.
3. **Planificación del Desarrollo**: consiste en la planificación del resto de fases del proceso de desarrollo. Se identifican las tareas a realizar, teniendo en cuenta las dependencias entre tareas y se estiman los tiempos de desarrollo, identificando los recursos y personal necesarios. Esta fase se apoya en **Diagramas de Pert** y **Gantt**.
4. **Diseño del Software**: durante esta parte se crea una arquitectura y un diseño para el software en base a los requisitos que se han definido. Esto incluye los modulos, la interfaz de usuario y los componentes, así como las relaciones entre estos. En esta fase hay que tener en cuenta la reducción de la complejidad, facilidad de mantenimiento, reducción de acoplamiento, ... y todas las decisiones de diseño que contribuyan a tener un software de calidad.
5. **Codificación**: en esta fase los desarrolladores escriben el código fuente para implementar el diseño y funcionalidades que se han especificado. El código debe ser claro, eficiente y fácil de mantener.
6. **Depuración y Pruebas**: después de implementarlo el software se somete a pruebas para identificar errores, problemas de rendimiento o comportamiento inesperados. Los tipos de pruebas que se pueden realizar en el software son la siguientes:
    - **Pruebas Unitarias**: se analizan elementos como clases, funciones, componentes o procesos, de forma individual e independiente del resto del sistema.
    - **Pruebas de Integración**: estas pruebas verifican que dos o más componentes funcionan correctamente entre ellos, evaluando sus interacciones.
    - **Pruebas de Sistema**: verifican que el sistema cumple con todos requisitos especificados. Se realizan en un entorno que simula el entorno de producción para validar el correcto funcionamiento del sistema.
    - **Pruebas de Rendimiento**: evalúa el sistema en condiciones de carga reales para garantizar la estabilidad y eficiencia deseadas.
    - **Pruebas de Seguridad**: se centran en identificar vulnerabilidades y posibles brechas en el software. Se realizan pruebas de penetración, análisis de vulnerabilidades y evaluación de riesgos para proteger al sistema contra amenazas internar y externas.
    - **Pruebas de Aceptación**: se realizan para validar que el software cumpla con los criterios de aceptación del cliente o el usuario final, centrándose en los requisitos de rendimiento y amigabilidad de la interfaz.

7. **Despliegue**: una vez que el software ha sido probado y validado, se despliega en el entorno donde vaya a ejecutarse o se distribuye a los usuarios finales.
8. **Mantenimiento**: después del despliegue, el software entra en fase de mantenimiento, durante la que se realizan actualizaciones, corrigen errores o añaden nuevas funcionalidades si esta previsto. Esta fase garantiza que el software siga siendo útil y funcional a lo largo del tiempo.

En la siguiente imagen podemos ver de una forma más gráfica todas estas etapas y su conexión:

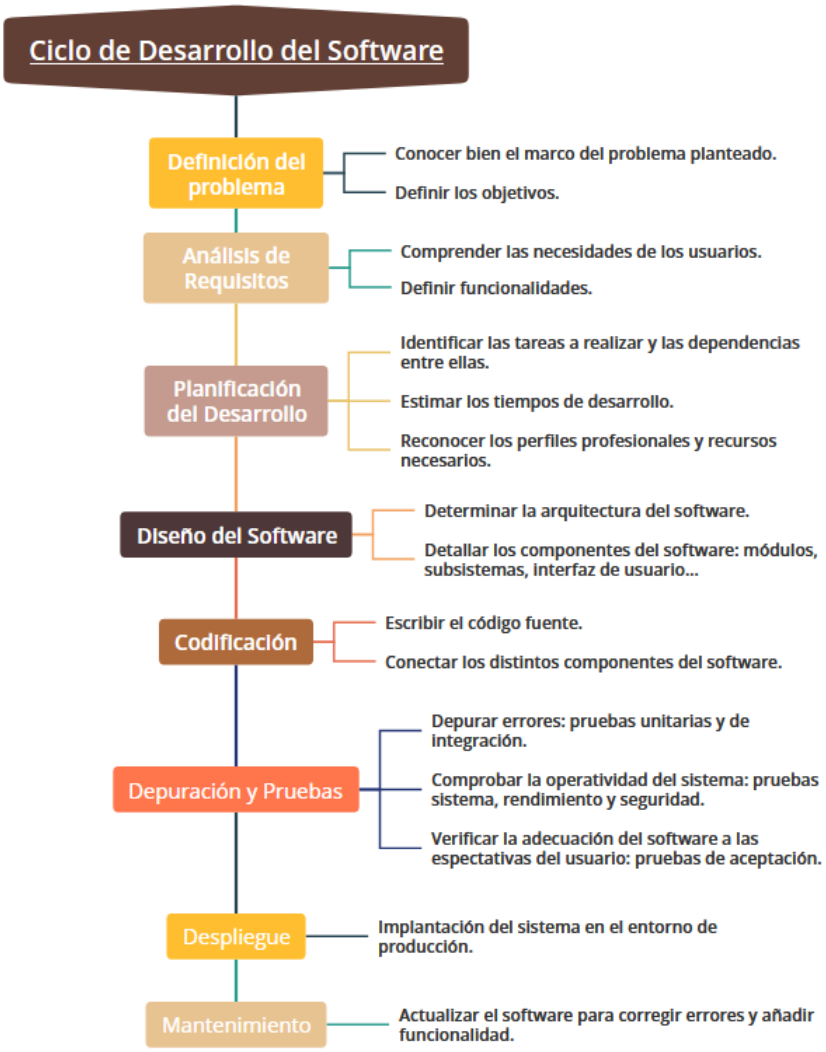


Figura 1.1: Ciclo de desarrollo del software

Como vemos, el desarrollo del software se realiza de forma muy estructurada y siguiendo un procedimiento establecido que garantiza la calidad del software y que estandariza el proceso de desarrollo.

Actualmente también podemos encontrar otros tipos de ciclos de desarrollo, que modifican ciertas etapas, como los que podemos encontrar en modelos como el **desarrollo ágil**, entre los que podemos encontrar **Scrum** o **Extreme Programming**.

Mención especial requiere el **machine learning** y su proceso de desarrollo, como vemos en el siguiente punto, que difiere un poco del proceso de desarrollo tradicional.

1.1.2.1. Machine Learning Los sistemas de IA pueden realizar tareas que normalmente requerirían de inteligencia humana, como el reconocimiento de voz, toma de decisiones, traducción de idiomas y la percepción visual. El objetivo principal de estos sistemas es inteligenes es pensar, aprender y adaptarse como si fueran humanos.

Para alcanzar este objetivo se hace uso de una rama de la programación de la Inteligencia Artificial conocida como **Machine Learning** (Aprendizaje Automático) que se centra en el desarrollo de algoritmos que permiten a las computadores **aprender a partir de datos** y hacer **predicciones** o **clasificaciones** basadas en esos datos. Este paradigma añade pasos extra en el proceso de desarrollo, ya que los modelos deben ser entrenados siguiendo ciertos pasos.

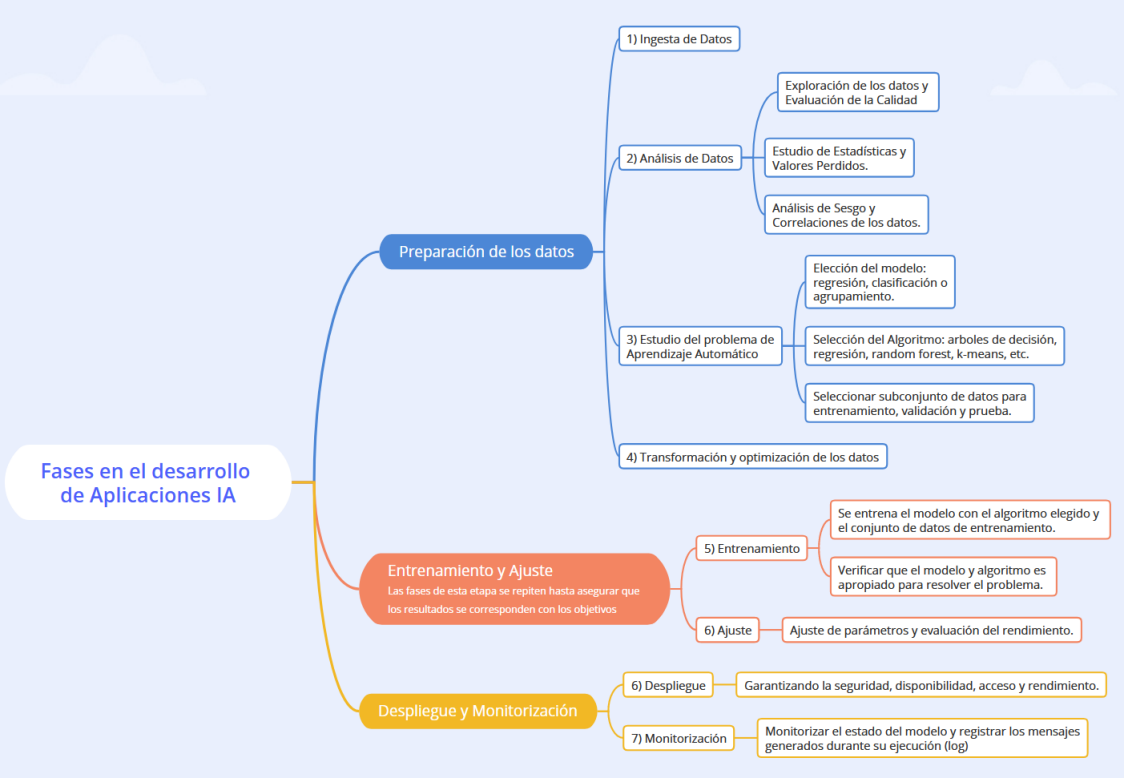


Figura 1.2: Etapas en el entrenamiento de un modelo de ML

En el aprendizaje automático se distinguen 3 fases principales con retroalimentación entre ellas:

1. **Preparación de los Datos**: consiste en obtener y preparar los datos para entrenar, validar y probar al modelo de Machine Learning. Los datos deben ser relevantes, de calidad y en cantidad suficiente para permitir el aprendizaje automático del modelo. Las principales tareas que comprende son:
   - **La Ingesta de Datos**: en esta tarea identifican las fuentes de datos interna y externas. Se recopilan, validan y estructuran los datos para su posterior análisis.
   - **Análisis y Transformación de los Datos**: en esta tarea se analizan los datos y se evalúa su relevancia para el problema planteado, se realizan estudios estadísticos, se tratan los valores perdidos, se identifican posibles sesgos y la correlaciones entre datos.
   - **Estudio de Problemas de Aprendizaje Automático**: en esta tarea se identifica el tipo de modelo más necesario (regresión, clasificación o agrupamiento), se selecciona el algoritmo más apropiado y se preparan los datos apropiados para el mismo.

- **Transformación y Optimización de los Datos**: suele ser necesaria la transformación de lo datos para el entrenamiento como paso previo, una vez identificadas las características de los mismos.

2. **Entrenamiento y Ajuste**: en esta fase se entrena el modelo con el algoritmo elegido y el conjunto de datos seleccionado, verificando la idoneidad de los datos de validación. Por último, se optimizan los hiperparámetros del entrenamiento para mejorar los resultados posibles.
3. **Despliegue y Monitorización**: se realiza el despliegue del sistema garantizando las condiciones de seguridad, disponibilidad y rendimiento. Además se registras la actividad del modelo para llevar un control de su funcionamiento.

Para ayudar en el proceso de entrenamiento del modelo, tenemos las **MLOps** (Machine Learning Operations), que son un conjunto de buenas prácticas con el objetivo de facilitar la implementación, el despliegue, la monitorización y el mantenimiento continuo de los modelos de aprendizaje automático en entornos de producción. Esto ayuda a mejorar la eficiencia, escalabilidad, calidad y confiabilidad de los sistemas de machine learning. [2]

1.2. Programación de Inteligencia Artificial

La **Inteligencia Artificial** (IA) tiene aplicaciones en prácticamente todos los sectores, desde la medicina, la astronomía y la agricultura, hasta la industria y el entretenimiento. Su versatilidad y capacidad de abordad una gama muy variada de desafíos la convierte en una herramienta muy poderosa en la resolución de problemas en la sociedad actual.

Las principales aplicaciones que tiene la IA son:

- **Clasificación y Reconocimiento de Patrones**: identificar patrones clasificar datos en categorías especificas, como la detección de Spam, la clasificación de imágenes médicas o datos de laboratorio, así como el reconocimiento de voz sistemas de asistentes virtuales.
- **Predicción y Análisis de Datos**: predecir tendencias y comportamientos según un conjunto de datos histórico, como el análisis predictivo para la toma de decisiones empresariales, la gestión de inventarios, la predicción de la demanda, detección de fraudes financieros o la modelización de fenómenos naturales y la predicción de desastres.
- **Optimización y Planificación**: encontrar soluciones óptimas para problemas complejos, como la planificación de rutas de entrega, la distribución y almacenamiento de mercancías, asignación de recursos, planificación de horarios, etc.
- **Recomendación Personalizada**: generar recomendaciones para los usuarios basados en su comportamiento, como la recomendación de contenido en plataformas de streaming, productos en tiendas, contenido en redes sociales, ...etc, con el objetivo de interactuar mejor con los usuarios.
- **Procesamiento del Lenguaje Natural (NPL)**: generar y comprender texto de forma natural, como en al traducción automática, generación de resúmenes, análisis de sentimientos, extracción de información de documentos o atención del cliente mediante chatbots
- **Visión por Computadora**: detectar y analizar imágenes y vídeos, como en la detección de objetos, el seguimiento de movimiento, reconocimiento facial, inspección automatizada de calidad o conducta autónoma.
- **Aprendizaje Automático**: desarrollar modelos que aprendan de datos para realizar tareas específicas, como diseñar nuevos fármacos analizando grandes cantidades de datos moleculares, de-

sarrollar materiales prediciendo sus propiedades, diseñar robots de propósito específico o incluso dotarlos de capacidad para adaptarse al entorno.

- **Generación de Contenido**: crear producción digitales innovadoras, como escenarios virtuales o personajes de videojuegos, contenidos educativos, música, construcción de modelos o simuladores.

Como vemos, la IA tiene un montón de aplicaciones, prácticamente en cualquier sector puede aplicarse de alguna u otra forma, ya sea en la gestión y planificación, o en la generación de contenidos, así como en el análisis de datos, ...etc, siendo, actualmente una de las tecnologías más versátiles que existen.

1.2.1. Elección del Lenguaje de Programación para IA

En la programación de la IA es fundamental la elección del **lenguaje de programación** en el que vamos a desarrollar el proyecto. Este debe cumplir ciertas **características**, entre las que encontramos:

- **Simplicidad**, para facilitar la escritura y el mantenimiento del código.
- **Capacidad de prototipado rápido** para generar versiones operativas en fases tempranas del desarrollo. El lenguaje debe ser capaz de ejecutar y probar código de manera ágil.
- **Legibilidad**, con un alto nivel de abstracción, que permite escribir código mas cercano al lenguaje y la lógica humana, para facilitar el trabajo colaborativo y la implementación de algoritmos previamente diseñados.
- **Existencia de bibliotecas y frameworks** para poder acelerar la generación de prototipos mediante la reutilización de código.
- **Comunidad de desarrollo** amplia que facilite compartir experiencias y soluciones a problemas ya resueltos.

Aunque como se ha comentado la elección del lenguaje de programación va ligada a las necesidades del proyecto y a las restricciones, hay ciertos **lenguajes** que por sus características son los **mas usados** en la programación de la IA, como pueden ser:

- **Python**: en la actualidad es el lenguaje de referencia para la IA. Se ha hecho muy popular en la programación de la IA debido a su sintaxis simple y legible, su ampliad disponibilidad de bibliotecas y frameworks de aprendizaje automático, como **TensorFlow**, **Keras**, **PyTorch** o **Scikit-Learn**.
- **R**: es uno de los lenguajes más utilizados en la estadística y el análisis de datos, lo que lo hace ideal para la investigación en ciencias de datos y aprendizaje automático. Tiene una amplia gama de paquetes y librerías enfocadas en el análisis estadístico y la visualización de datos, siendo una opción muy popular en el modelado predictivo y exploración de datos en IA.
- **Java**: una de las principales características de Java es su portabilidad, en parte gracias a la Java Virtual Machine (JVM) y la posibilidad de convertir código fuente Java en bytecode. Aunque no es tan empleado como Python o R, es empleado para del desarrollo de sistemas de IA empresariales y aplicación a gran escala, mediante el uso de la librería **Weka Deeplearning4j**.
- **C++**: este lenguaje de programación orientado a objetos y compatible con C, ofrece características como soporte de clases, polimorfismo, plantillas y una biblioteca estándar muy potente y flexible. Es conocido por su velocidad y eficiencia, por lo que es usado en aplicaciones de IA que requieren de un rendimiento óptimo, como en sistemas embebidos, procesamiento de señales y visión por computador. Bibliotecas como **OpenCV** y **TensorFlow** ofrecen soporte para este lenguaje.

- **Julia**: este lenguaje quizá es el menos conocido de todos los mencionados, siendo un lenguaje diseñado para el computo científico y numérico, lo que lo hace adecuado para aplicaciones de IA que involucran cálculos intensivos y modelado matemático. Tiene un sintaxis similar a Pytho o R pero con un rendimiento similar a C++.

Estos son los lenguajes más empleados en la programación de IA, pero no los únicos, ya que como hemos dicho, su elección depende mucho del proyecto donde vayamos a emplearlos. Si tienes más curiosidad por otros lenguajes de programación, puedes consultar el indice TIOBE, que mide la popularidad de los lenguajes de programación.

1.2.2. Lenguajes de Marcas e IA

Los lenguajes de marcas están compuestos de un conjunto de etiquetas y reglas de uso, y por normal general, a diferencia del resto de lenguajes de programación, se utilizan únicamente para estructurar documentos y organizar los datos para su posterior procesamiento. Algunos de los lenguajes de marcas más empleados son:

- **HTML** (HyperText Markup Language): este lenguaje es la base del desarrollo web y se emplea para especificar que contenidos y con que estructura aparecen en una página web.
- **XML** (eXtensible Markup Language): es un lenguaje de marcado de propósito general y que permite la creación de etiquetas propias. Su principal propósito es estructurar y compartir datos entre diferentes sistemas.
- **JSON** (Javascript Object Notation): no es un lenguaje de marcas en el sentido tradicional como HTML o XML. Sin embargo, JSON es un formato ligero basado en texto que al igual que XML se utilizar para estructurar datos y compartirlos entre sistemas. Los datos se estructuran en formato çlave:valorz listas ordenadas de valores. Es uno de los formatos de intercambio de información más empleados, especialmente en el desarrollo web.

Aunque en principio no parece que haya una relación muy directa entre los lenguajes de marcado y la IA, realmente son ampliamente utilizados. **HTML** suele emplearse para diseñar interfaces de usuario o como fuente de dato Web Scraping para entrenar modelos de IA.

XML suele emplearse para compartir datos entre diferentes componentes del sistema, así como para establecer el funcionamiento del sistema mediante archivos de configuración.

Por su parte, **JSON** se utiliza para compartir información entre elementos del sistema. Por ejemplo, las APIs del sistema suelen proporcionar los resultados a los usuarios en este formato. También se emplea para construir bases de datos no relacionales.

1.2.3. Computación en la Nube

La **computación en la nube** es modelo que permite el acceso ubicuo, cómodo y a la carta a un conjunto de recursos informático configurables, como redes, servidores, almacenamiento, aplicaciones y servicios, que pueden ser rápidamente provisionados y liberados con un mínimo de esfuerzo de gestión o de interacción entre el proveedor de servicios.

Este tipo de modelo ofrece ciertas **ventajas** respecto a los modelos tradicionales de computación:

- **Bajo Demanda y Ahorro de Costes**: la computación en la nube ofrece los recursos necesarios en el momento oportuno, sin que el usuario necesite aprovisionar o mantener los recursos que

necesita. El pago se realiza por el consumo de servicios que se ha realizado, ya sea en horas, trafico, espacio, ...etc.

- **Agilizar y Elasticidad**: la computación en la nube permite escalar los recursos de forma rápida y eficiente en función de la carga del sistema. Se puede monitorizar el sistema e incrementar o decrementar la infraestructura en función de las necesidades.
- **Facilidad para el Despliegue**: se flexibilizan los mecanismos para mover un modelo prototipo a un entorno productivo, proporcionando herramientas para la automatización del despliegue.

Si tienes interés para aprender más sobre la nube o te ha quedado alguna duda puedes consultar este vídeo de LearnFreen sobre la nube.

Glosario

bytecode Es un lenguaje intermedio un poco más abstracto que el lenguaje máquina. En caso de Java, este bytecode será interpretado por la máquina virtual JVM. 5

sesgos El sesgo en inteligencia artificial son las tendencias o prejuicios sistemáticos que pueden influir en el comportamiento o decisiones del modelo de IA. 9

chatbots Programa informático que usa IA y NLP para comprender las preguntas de los clientes y automatizar las respuestas simulando una conversación humana. 10

Web Scraping Es una técnica que se emplea para extraer datos de una página web de forma automatizada, descargando los documentos HTML de una web y extrayendo la información. 12

Referencias

[1] Wikipedia - Programación Funcional.
https://es.wikipedia.org/wiki/Programaci%C3%B3n_funcional.

[2] AWS - MLOps.
https://aws.amazon.com/es/what-is/mlops/.